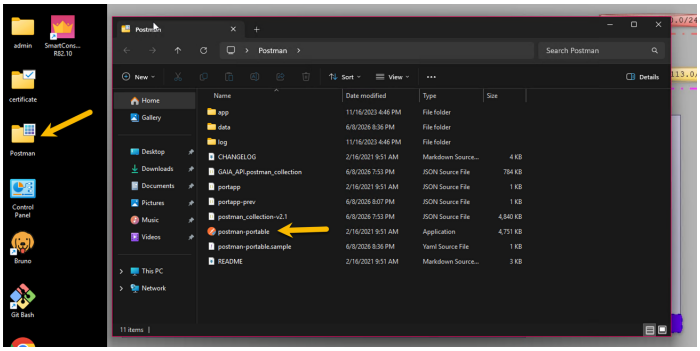
Postman & the Management API

Objectives

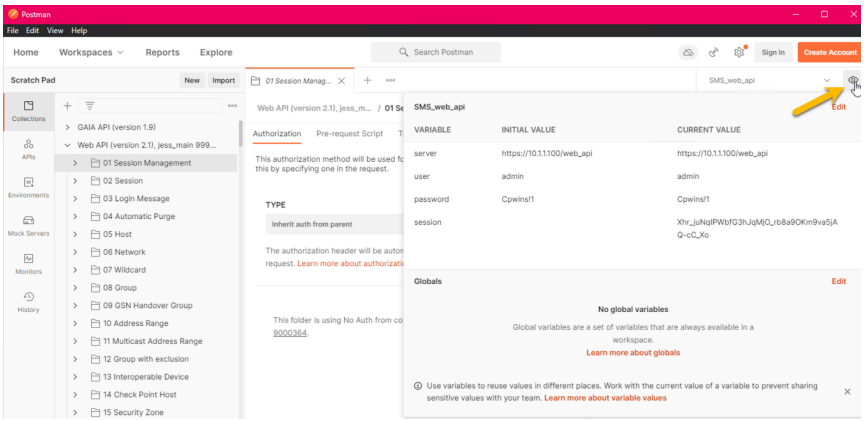
1. Configure Postman with an environment and import the R82.10 Management API collection.
2. Drive the session lifecycle (login → add → publish → logout) entirely from Postman.
3. Create groups and a custom policy package with named sections and rules via API.
4. Use add-objects-batch for bulk operations and show-validations to gate publish.
5. Answer the CISO question: "We don't allow Postman cloud — what do we use instead?"

Pre-Flight

1. On the jump server, open **Postman**. Skip any sign-in prompt — you will work locally for this lab.



2. Review the lab environment: click the **eye icon** (top right) →. Review the variables used by this collection.

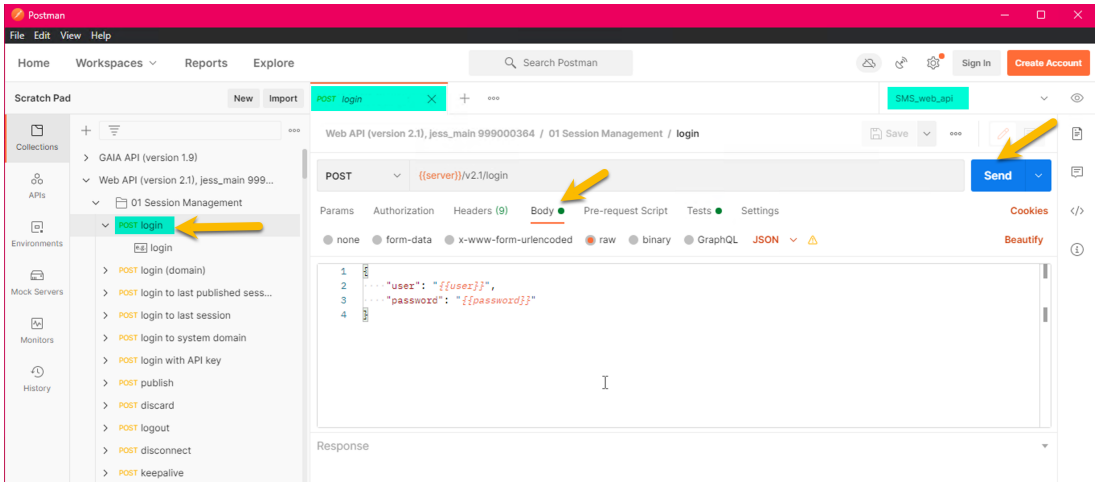


Task 1: Session Lifecycle (10 Minutes)

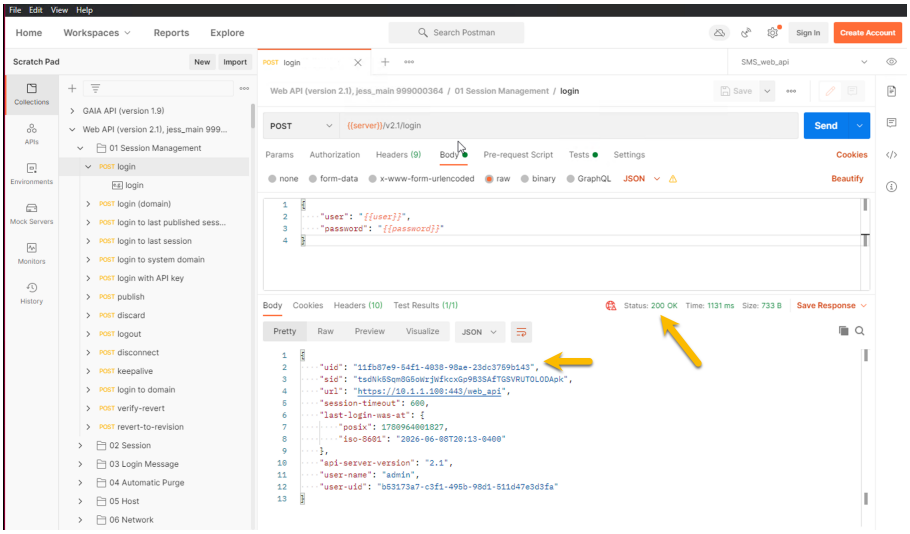
The same four-step session pattern from Lab 1, but driven from the Postman UI. Every other task in this lab will repeat this lifecycle.

1a — Login

In the imported collection, expand **01 Session Management** (or use the filter at the top). Click **login**, open the **Body** tab, review its contents.

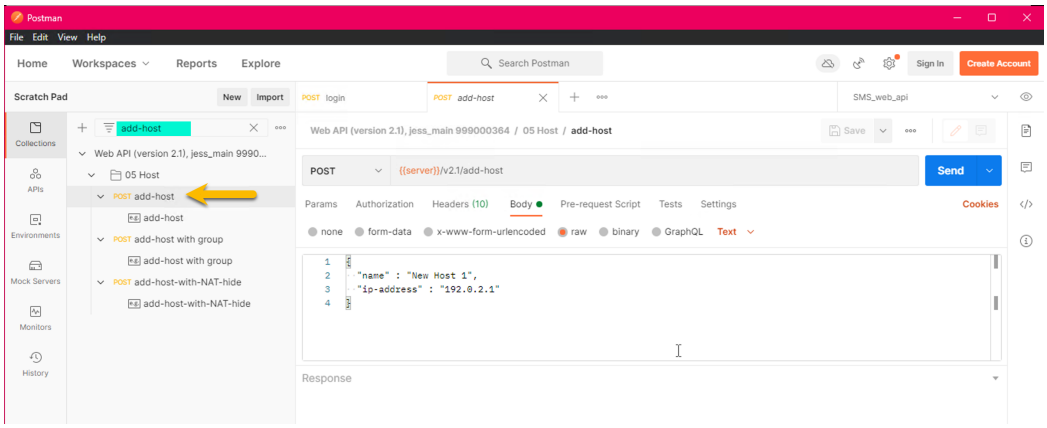


Click **Send**. Confirm: status `200`, response body contains `sid`. The collection's **Tests** tab on the login request auto-stores the `sid` into the environment variable. Open the environment editor (eye icon) to confirm `sid` is now populated.



1b — Add a host, publish, logout

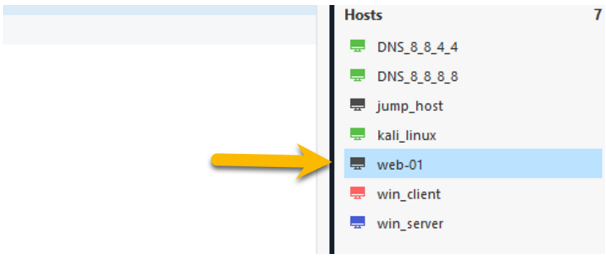
Filter the collection for `add-host`.



Set the body:

```
{ "name": "web-01", "ip-address": "10.20.30.1" }
```

Send. Confirm 200. Then filter for `publish`, Send (body is `{}`). Then filter for `logout`, Send. Verify in SmartConsole that `web-01` appears under **Network Objects → Hosts**.

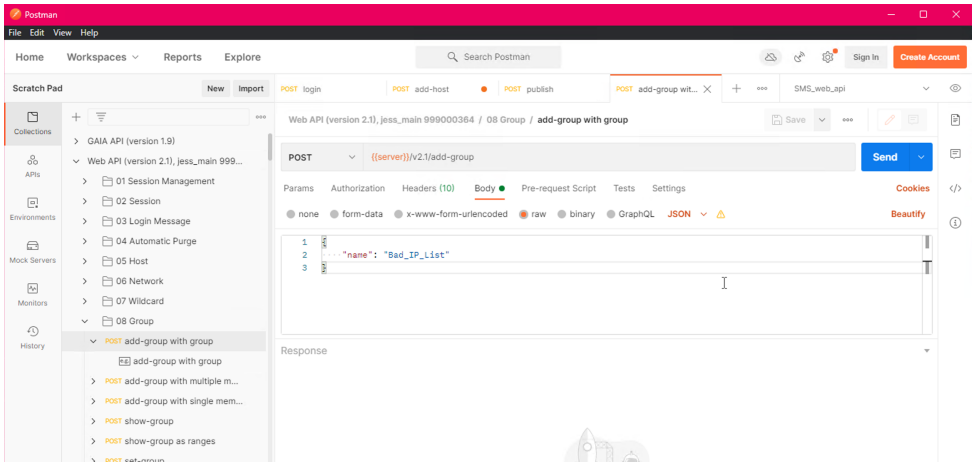


The pattern. Every state change in the rest of this lab follows the same shape: login → one or more requests against `{{sid}}` → publish → logout.

Task 2: Create Two Groups (15 Minutes)

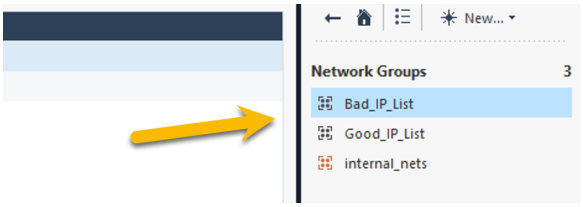
Groups are the simplest "create" object on the API. Two of them now, using the same lifecycle from Task 1. The first call is login — your previous session is closed.

1. `login` → Send (body unchanged from Task 1).
2. `add-group` with body `{ "name": "Bad_IP_List" }`. Send. Confirm 200.



3. `add-group` with body `{ "name": "Good_IP_List" }`. Send.
4. `publish` → Send. `logout` → Send.

In SmartConsole → **Network Objects** → **Groups**, both groups should appear.



Question. A group with zero members is a valid object. Why is that useful in practice?

Answer. It is a stub: a policy can reference the empty group today, and an automation (a threat-feed integration, a SOAR webhook, a SIEM enrichment job) can populate the members later without touching the policy itself. This pattern is how dynamic IoC / blocklist integrations work in production.

Task 3: Policy Package, Sections, and Rules (20 Minutes)

A new policy package called `APITraining`, with two named sections and one rule per section. End-to-end policy authoring without ever opening SmartConsole.

3a — Create the package

Login (new session). Filter for `add-package`. Body:

```
{
  "name": "APITraining",
  "comments": "Lab 3 demo package",
  "access": true,
  "threat-prevention": true
}
```

Send. The response includes an `access-layers` array. Note the layer name (typically `APITraining Network`) — you will reference it in 3b and 3c.

3b — Add two named sections

Filter for `add-access-section`. Body for the first section:

```
{ "layer": "APITraining Network", "name": "Admin Access", "position": "top" }
```

Send. Body for the second section:

```
{ "layer": "APITraining Network", "name": "API Access", "position": "bottom" }
```

3c — Add one rule to each section

Filter for `add-access-rule`. First rule (under Admin Access):

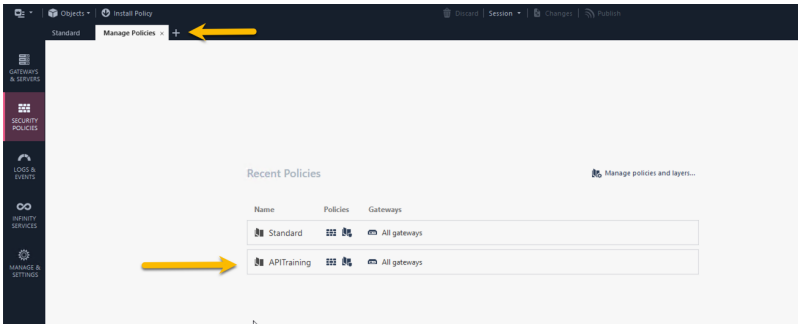
```
{
  "layer": "APITraining Network",
  "name": "Allow SSH/ICMP for admins",
  "position": { "below": "Admin Access" },
  "source": "Any", "destination": "Any",
  "service": ["ssh", "icmp-proto"],
  "action": "Accept"
}
```

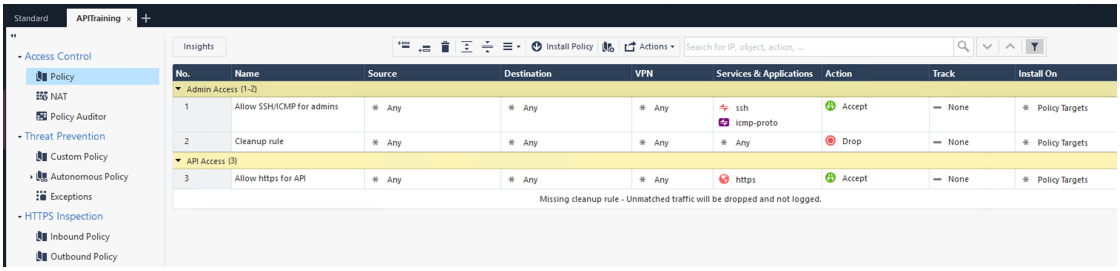
Second rule (under API Access):

```
{ "layer": "APITraining Network", "name": "Allow https for API",
  "position": { "below": "API Access" },
  "source": "Any", "destination": "Any",
  "service": "https", "action": "Accept" }
```

`publish`, then `logout`.

In SmartConsole → **Security Policies** → **APITraining**: two sections, one rule each.





Task 4: Batch Operations & Validation (15 Minutes)

Instead of N round-trips for N objects, the Management API exposes `add-objects-batch` — one call, many objects, mixed types. Paired with `show-validations` as a pre-publish gate, this is the modern bulk pattern.

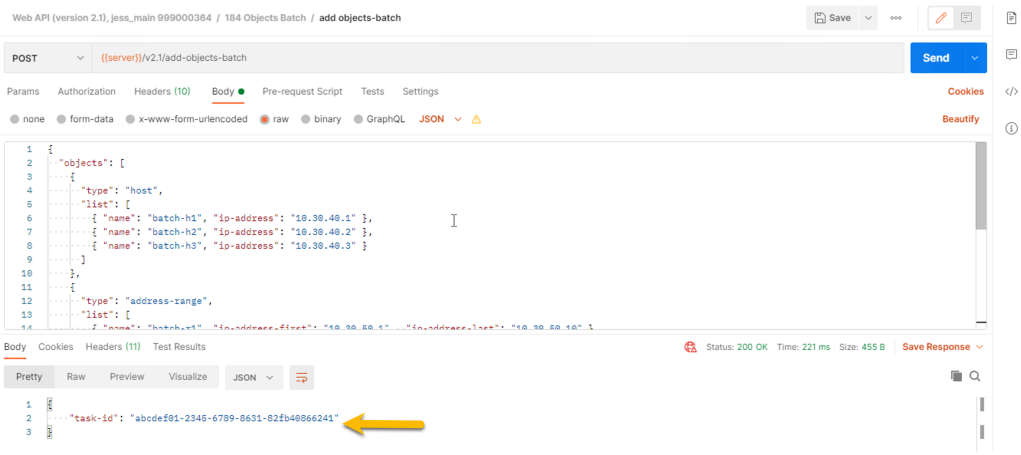
4a — Bulk-create nine objects in one call

1. Login. Filter for `add-objects-batch`. Body:

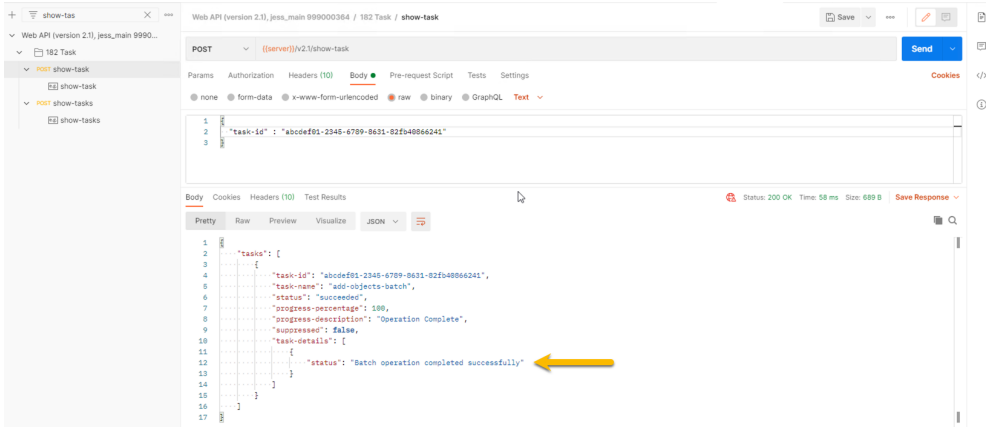
```
{
  "objects": [
    {
      "type": "host",
      "list": [
        { "name": "batch-h1", "ip-address": "10.30.40.1" },
        { "name": "batch-h2", "ip-address": "10.30.40.2" },
        { "name": "batch-h3", "ip-address": "10.30.40.3" }
      ]
    },
    {
      "type": "address-range",
      "list": [
        { "name": "batch-r1", "ip-address-first": "10.30.50.1",  "ip-address-
last": "10.30.50.10" },
        { "name": "batch-r2", "ip-address-first": "10.30.50.11", "ip-address-
last": "10.30.50.20" },
```

```
        { "name": "batch-r3", "ip-address-first": "10.30.50.21", "ip-address-
last": "10.30.50.30" }
      ]
    },
    {
      "type": "service-tcp",
      "list": [
        { "name": "batch-s1", "port": "9001" },
        { "name": "batch-s2", "port": "9002" },
        { "name": "batch-s3", "port": "9003" }
      ]
    }
  ]
}
```

2. Send. The response is a task ID. We will use a separate API call to get more async details.

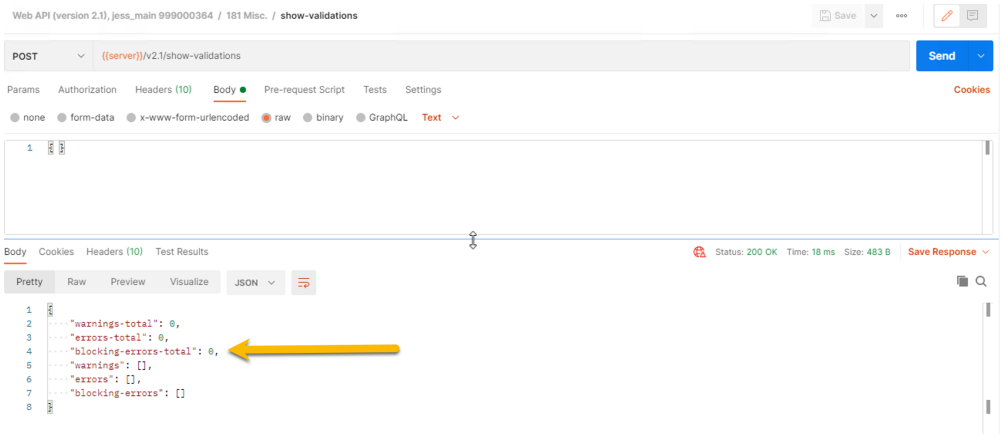


3. Filter for `show-task`, replace the body with the task ID above and click Send.



4b — Validate before publishing

4. Filter for `show-validations`. Body is `{}`. Send. The response lists every pending change in the current session plus any warnings or errors.



The discipline. Every multi-step API change in production should call `show-validations` before publish. Never blind-publish a multi-step session.

5. `publish`, `logout`. Verify in SmartConsole: 9 new objects.

Cleanup

The lab leaves behind one host, two groups, one policy package, and nine batch objects. Run the steps below to remove them so the next cohort starts clean.

Step 1. Send `login` with the usual admin / Cpwins!1 body. New `sid` populates automatically.

Step 2. Filter for `delete-objects-batch`. Body removes all nine batch objects in one call:

```
{
  "objects": [
    {
      "type": "host",
      "list": [
        { "name": "batch-h1" },
        { "name": "batch-h2" },
        { "name": "batch-h3" }
      ]
    },
    {
```

```
        "type": "address-range",
        "list": [
          { "name": "batch-r1" },
          { "name": "batch-r2" },
          { "name": "batch-r3" }
        ]
      },
      {
        "type": "service-tcp",
        "list": [
          { "name": "batch-s1" },
          { "name": "batch-s2" },
          { "name": "batch-s3" }
        ]
      }
    ]
}
```

Step 3. Filter for `delete-host`. Body:

```
{ "name": "web-01" }
```

Step 4. Filter for `delete-group`. Send twice — once per group:

```
{ "name": "Bad_IP_List" }
{ "name": "Good_IP_List" }
```

Step 5. Filter for `delete-package`. Body:

```
{ "name": "APITraining" }
```

Step 6. Send `publish`, then `logout`.

Verify. In SmartConsole: no lab artifacts under **Network Objects**, and `APITraining` no longer appears under **Security Policies**.